



1.1. Imputation of Missing Values

Missing data is everywhere. Good analysts know how to deal with it.

Previous Section:

 [0. Data Analytics](#)

Every real-world dataset has imperfections. Some values are entered incorrectly, some are duplicated, and some are simply **missing**.

In fact, **missing values** are one of the first problems you will encounter in almost every data analytics project. Before building charts, running machine learning models, or discovering interesting insights, we first need to make sure our data is complete enough to work with.

In this section, we'll learn what missing values are, why they happen, and several common techniques to handle them using **Pandas**.

To make things interesting, instead of downloading an existing dataset, we'll generate our own synthetic student dataset using **NumPy**. The dataset contains 20 students with the following information:

- **Name** (randomly selected from a list of names)
- **Age** (between 18 and 20)
- **Major** (AI, IT, CSC, or SW)
- **GPA** (between 2.0 and 4.0)

After generating the dataset, we'll intentionally replace about **20% of the values** in the **Age**, **Major**, and **GPA** columns with missing values. This gives us a realistic dataset to practice cleaning and imputation techniques throughout the chapter.

Python Code

```

import numpy as np
import pandas as pd

# Reproducible randomness
np.random.seed(42)

names = [
    "James", "Chris", "Adam", "Marian", "Peter",
    "Kevin", "Linda", "Emma", "Sophia", "Daniel",
    "Noah", "Olivia", "Lucas", "Grace", "Henry",
    "Liam", "Ava", "Mia", "Nathan", "Chloe"
]

majors = ["AI", "IT", "CSC", "SW"]

n = 20

df = pd.DataFrame({
    "Name": np.array(names),
    "Age": np.random.randint(18, 21, n),
    "Major": np.random.choice(majors, n),
    "GPA": np.round(np.random.uniform(2.0, 4.0, n), 2)
})

# Introduce 20% missing values into selected columns
for column in ["Age", "Major", "GPA"]:
    missing_idx = np.random.choice(df.index, int(0.2 * n),
    replace=False)
    df.loc[missing_idx, column] = np.nan

print(df.head(10))

```

The first 10 samples:

index	Name	Age	Major	GPA
0	James	NaN	SW	2.91
1	Chris	18.0	SW	NaN

index	Name	Age	Major	GPA
2	Adam	NaN	NaN	2.4
3	Marian	20.0	AI	NaN
4	Peter	NaN	SW	NaN
5	Kevin	18.0	IT	NaN
6	Linda	NaN	IT	3.22
7	Emma	19.0	AI	2.34
8	Sophia	20.0	SW	2.13
9	Daniel	20.0	NaN	3.9

The “NaN” means “Not a Number” and is classified as a numeric float type

Get dataset information:

```
# Get Column Non-Null Count Dtype as DataFrame
df_info = pd.DataFrame(df.dtypes).rename(columns={0: 'column
n_type'})
df_info['Non-Null Count'] = df.count()
print(df_info[['Non-Null Count', 'column_type']])
```

index	Non-Null Count	column_type
Name	20	object
Age	16	float64
Major	16	object
GPA	16	float64

This means for the index Age, Major, GPA. Each of them has 16 values that are unknown. Making it 48 missing values total.

1. What are Missing Values?

A **missing value** is exactly what it sounds like—a piece of information that should exist but doesn’t. Instead of containing a valid value, the entry is empty or unknown.

Consider this simple student dataset.

Name	Age	Major	GPA
James	12	AI	3.5
Chris	Missing	CSC	3.2
Adam	15	Missing	2.8
Marian	Missing	AI	2.3
Peter	18	AI	2.1
Kevin	19	IT	Missing

Even in this tiny table, we already have several missing values. Imagine working with millions of records—missing data quickly becomes a major challenge.

1.1. Why do missing values happen?

There are many reasons why data may be incomplete:

- Someone skips a question in a survey.
- A typo or data entry mistake leaves a field blank.
- A sensor temporarily fails to record a measurement.
- Different datasets are merged together, but some columns don't exist in every source.

In short, missing values are a normal part of working with real-world data.

1.2. Why are missing values a problem?

Ignoring missing values can lead to several issues:

- Statistical calculations become inaccurate.
- Analysis results can become biased.
- Machine learning models may perform poorly—or fail to run entirely.
- Conclusions drawn from the data become less reliable.

For these reasons, **handling missing values is one of the first steps in data cleaning**. Before we can trust our analysis, we need to decide what to do with the missing information.

2. Types of Missing Values

Not all missing values happen for the same reason. Understanding **why** data is missing often helps us choose the best way to handle it.

There are three common types of missing values.

- **Missing Completely at Random (MCAR):** This is the easiest case.
 - The missing values have **no relationship with any other variable** in the dataset. Every record has the same chance of being missing.
 - For example, imagine a system crashes while saving data, causing a few random GPAs to disappear. The missing values are purely accidental and do not follow any pattern.
- **Missing at Random (MAR):** Here, the missing values **do follow a pattern**, but that pattern depends on **other variables**, not on the missing value itself.
 - For example, suppose a survey asks whether people drink alcohol. Younger participants may skip this question more often than older participants. The missing responses are related to **Age**, not to whether they actually drink.
 - Once we understand the relationship, we can often make better decisions when filling in the missing values.
- **Missing Not at Random (MNAR):** This is usually the most difficult situation. The missing value depends on the value that is actually missing.
 - For example, people with a very high body weight may choose not to report their weight in a survey. In this case, the missing values themselves carry useful information—they are missing **because of the value they contain**.
 - Since the reason for the missing data is hidden, MNAR data is often the hardest to analyze and requires extra care.

In practice, you don't always know which type of missing value you're dealing with. However, understanding these three categories helps explain why there is **no single "correct" way** to handle missing data. The best solution depends on both the dataset and the reason the values are missing.

3. Common Methods for Handling Missing Values

There is no single “best” way to deal with missing values. The right approach depends on **how much data is missing**, **what the feature represents**, and **how the data will be used later**.

Some of the most common approaches include:

- **Remove records or features** that contain too many missing values.
- **Fill (impute) the missing values** using simple statistical measures.
- **Forward Fill / Backward Fill**, which are commonly used for sequential or time-series data.
- **Advanced imputation methods**, such as Regression, Classification models, K-Nearest Neighbors (KNN), or Multiple Imputation by Chained Equations (MICE).

In this section we'll focus on the first two approaches since they are the most commonly used and easiest to understand.

Method 1. Removing Missing Values

Sometimes the simplest solution is to remove the incomplete data.

If only a few records contain missing values, removing those rows may have little impact on the overall dataset. Likewise, if an entire feature contains mostly missing values, it may not provide enough useful information and can simply be removed.

This approach is straightforward, but it should be used carefully. Removing too much data can reduce the amount of information available for analysis and may even introduce bias if the missing values are not random.

Method 2. Imputing Missing Values

Instead of removing data, we can **replace the missing values with reasonable estimates**.

Consider the following dataset after filling in the missing values.

Name	Age	Major	GPA
James	12	AI	3.5
Chris	Missing	CSC	3.2
Adam	15	Missing	2.8
Marian	Missing	AI	2.3
Peter	18	AI	2.1
Kevin	19	IT	Missing

Suppose the original dataset contained several missing values. We could replace them using simple statistics:

- **Mean of Age:** $\frac{12+15+18+19}{4} = 16$
- **Median of GPA:** 2.8
- **Mode of Major:** *AI*

As a result:

Name	Age	Major	GPA
James	12	AI	3.5
Chris	16	CSC	3.2
Adam	15	AI	2.8
Marian	16	AI	2.3
Peter	18	AI	2.1
Kevin	19	IT	2.8

These three statistics are the most common choices for imputation.

- **Mean** is usually used for continuous numerical features.
- **Median** works well for integer-valued or skewed numerical features because it is less affected by extreme values.
- **Mode** is the natural choice for categorical or ordinal features since it replaces missing values with the most common category.

Choosing the Right Imputation Method

The choice of imputation method depends on the feature itself. For example:

- If the feature is **categorical** (such as Major or Gender), using the **mean** or **median** makes no sense. The **mode** should be used instead.
- For **continuous numerical features** (such as GPA or Salary), the **mean** is often a good choice because it preserves the overall average.
- For features like **Age**, the **median** is often preferred since it naturally produces a realistic whole number and is less affected by unusually young or old values. Even if the mean is used, rounding the result is usually sufficient.

There is no universal rule. Understanding the meaning of the data is just as important as knowing the available techniques.

4. Imputing Missing Values with Pandas

Pandas makes missing value imputation extremely simple.

- First, let's see how many missing values exist in each column.

```
print(df.isnull().sum())
```

Alternatively, you can use `df.isnull().sum()`

- If we decide to **remove** incomplete records, we can simply use:

```
# Remove rows containing at least one missing value
df_drop = df.dropna()
```

- Or, if we want to **fill** the missing values:

```
# Fill numerical columns
df["Age"] = df["Age"].fillna(round(df["Age"].median()))
df["GPA"] = df["GPA"].fillna(df["GPA"].mean())

# Fill categorical column
df["Major"] = df["Major"].fillna(df["Major"].mode()[0])

print(df.head(10))
```


Notice that `mode()` returns a Series because there may be multiple modes. Therefore, we use `[0]` to obtain the first (and usually only) most frequent value.

Summary

This section covers the problem of missing, or unknown values in data analysis and how we deal with it.

- Missing values can either be removed or replaced.
- Removing rows or columns is simple but may discard useful information.
- Imputation replaces missing values with reasonable estimates instead of deleting data.
- Mean is commonly used for continuous numerical data, median for integer or skewed numerical data, and mode for categorical data.
- Pandas provides convenient functions such as `dropna()` and `fillna()` that make handling missing values quick and straightforward.

To practice further, complete Lab 01 of the Data Analytics Hands-On Lab.

 [Lab 07: Data Analytics with Python](#)

References

https://pandas.pydata.org/docs/user_guide/missing_data.html

<https://www.geeksforgeeks.org/machine-learning/what-is-data-imputation/>

Next Section:

 [1.2. Handling Unique and Duplicate Values](#)